

面向 LLM 智能体的组合式技能路由： 分解、检索与组合

匿名

摘要

大语言模型（LLM）智能体日益依赖外部技能（skill）——可复用的工具规范——但现实任务往往需要组合多项技能，而非仅选择单一技能。我们将此形式化为**组合式技能路由**（Compositional Skill Routing）问题：给定一个复杂用户查询和一个大规模技能库，将查询分解为原子子任务，为每个子任务检索合适的技能，并组合生成可执行计划。我们提出 **SKILLWEAVER**——一个分解-检索-组合框架，结合 LLM 任务分解器、基于双编码器的技能检索器（FAISS 索引）以及依赖感知的 DAG 规划器。为支持评估，我们构建了 **COMP-SKILLBENCH**，包含 300 个组合式查询，覆盖 2,595 个技能和 17 个功能类别。实验表明任务分解质量是主要瓶颈：oracle 分解可达 99.5% 的技能召回率，而标准 LLM 分解在类别级别仅达 33.9%。为此，我们提出技能感知分解（SAD），一种检索增强方法，使分解器获知可用技能信息，将类别召回率提升至 54.2%（+60%），在 5 个模型（7B–72B）上均有效；一个 7B 模型配合 SAD 的表现优于不使用 SAD 的 72B 模型。**SKILLWEAVER** 相较于暴露所有工具，上下文窗口消耗减少超过 99%。

1 引言

大语言模型的智能体范式已从单轮生成扩展到工具使用、规划和多步任务执行 [Schick et al., 2023, Qin et al., 2023, Patil et al., 2024]。现代 LLM 智能体中一个关键的架构模式是使用技能（skill）：模块化、可复用的工具规范，定义特定能力及其调用时机和方式 [Anthropic, 2025]。随着技能库规模增长——社区贡献的技能已达数千——一个基本的路由问题浮现：给定用户查询，智能体应调用哪些技能？

先前工作主要将技能路由视为单技能选择问题，目标是为查询匹配单个最佳技能 [Zheng et al., 2025]。然而，现实用户查询往往需要多项技能协同工作。例如，查询“下载数据集、对其进行转换并生成可视化报告”至少需要三种技能：用于下载的 API 客户端、用于转换的数据处理器以及用于生成报告的可视化工具。

我们将此形式化为**组合式技能路由**问题（图 1）：给定复杂查询 q 和技能库 S ，生成有序技能序列 $[s_1, s_2, \dots, s_k]$ ，使其协同完成查询，其中每个 s_i 处理一个原子子任务。该形式化捕捉了现实任务的组合性质，同时保持了基于技能架构的模块化优势。

我们提出 **SKILLWEAVER**，一个三阶段框架：

1. **分解**（Decompose）：基于 LLM 的任务分解器将复杂查询拆分为原子子任务，每个子任务恰好需要一项技能。

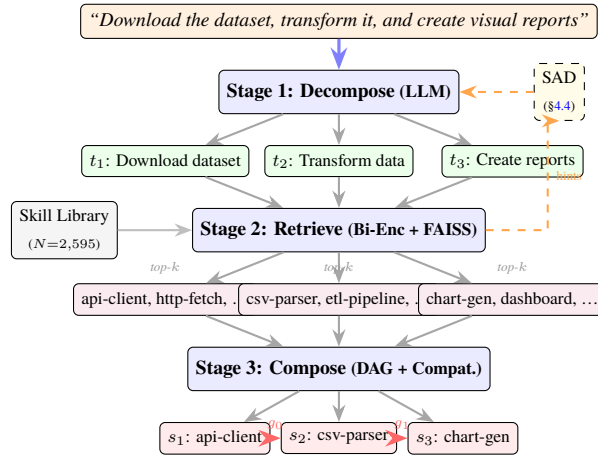


图 1: SKILLWEAVER 系统概览。查询被分解为子任务，每个子任务通过双编码器检索匹配技能，最后组合为 DAG。虚线箭头：SAD 反馈环路 (§4.4)。

2. **检索 (Retrieve)**: 双编码器检索器利用技能元数据上的语义相似度，为每个子任务识别候选技能。
3. **组合 (Compose)**: 兼容性感知的规划器为每步选择最优技能并考虑技能间兼容性，生成可执行 DAG。

为评估组合式技能路由，我们构建了 COMP-SKILLBENCH——首个针对该任务的基准。COMP-SKILLBENCH 包含 300 个组合式查询，覆盖 2,595 个精选技能和 17 个功能类别，具有真实技能链和三个难度级别。技能来源于 212 个公开 GitHub 仓库，经去重确保质量。

我们的实验得出如下关键发现：

- **分解是瓶颈**: 使用 oracle 分解，检索器达到 99.5% Recall@1 和完美的类别准确率。标准 LLM 分解降至 33.9% CatR@1，表明分解是关键前沿。
- **技能感知分解弥合差距**: 我们提出的技能感知分解 (SAD) 为分解器提供检索到的技能上下文，将 CatR@1 从 33.9% 提升至 54.2% (+60%)，且无需更换底层模型。
- **元数据即可满足检索**: 仅使用元数据的编码与包含正文的编码表现相当或更优，说明简洁的技能元数据携带了最具辨别力的信号。

2 相关工作

工具选择与路由。 LLM 的工具选择已通过 API 检索 [Patil et al., 2024, Qin et al., 2023]、工具文档匹配 [Hao et al., 2024] 和层级路由 [Zheng et al., 2025] 等方式进行研究。AnyTool [Du et al., 2024] 使用自反思机制导航层级 API 池，CRAFT [Yuan et al., 2025] 通过检索为每个查询创建专门的工具集。SkillRouter [Zheng et al., 2025] 与我们的工作最为接近，引入了面向单技能路由的双编码器方法。然而，上述方法均独立地为每个查询选择工具，未对组合式任务联合优化分解、检索和技能间兼容性。

工具增强的 LLM 基准。 工具使用评估受到日益关注。API-Bank [Li et al., 2023] 和 ToolQA [Zhuang et al., 2024] 分别对单工具和多步工具使用进行基准测试。ToolSandbox [Lu et al., 2024] 提供有状

态的评估框架，MetaTool [Huang et al., 2024] 关注是否使用工具的决策。TaskBench [Shen et al., 2023] 评估多步任务，但基于固定工具集而非从大规模库中检索。COMP SKILL BENCH 是首个专门针对数千技能之上组合式路由的基准。

任务分解与规划。 基于 LLM 的任务分解已通过提示策略 [Wei et al., 2022, Zhou et al., 2022, Khot et al., 2023]、规划框架 [Huang et al., 2022, Wang et al., 2023] 和智能体系统 [Yao et al., 2023, Shinn et al., 2023] 进行探索。Press et al. [2023] 识别了语言模型中的“组合性差距”，为基于分解的方法提供了动机。TPTU [Ruan et al., 2024] 和 AgentFlan [Chen et al., 2024] 研究联合任务规划与工具使用。我们的贡献是一种新颖的技能感知分解方法，分解器由技能库本身提供信息，形成检索增强的反馈环路。

检索增强生成。 基于双编码器的稠密检索 [Karpukhin et al., 2020, Ni et al., 2022] 已广泛应用于知识密集型 NLP 任务。我们将此范式适配于技能检索，并进一步扩展至为上游分解阶段提供信息——利用检索结果作为分解指引。

3 问题形式化

技能库。 技能库 $\mathcal{S} = \{s_1, \dots, s_N\}$ 包含 N 个技能。每个技能 s_i 是一个元组 (n_i, d_i, b_i, C_i) ，其中 n_i 为名称， d_i 为自然语言描述， b_i 为完整规范正文（指令、示例、配置）， $C_i \subseteq \mathcal{C}$ 为功能分类体系 $\mathcal{C}$ 中的类别集合。

组合式技能路由。 给定需要多种能力的复杂查询 q ，目标是生成：

1. 分解 $D(q) = [t_1, \dots, t_K]$ ，包含 K 个原子子任务。
2. 技能指派 $\sigma : [t_1, \dots, t_K] \rightarrow \mathcal{S}^K$ ，将每个子任务映射到一个技能。
3. 执行计划（DAG） $G = (V, E)$ ，指定步骤间的依赖关系。

组合式路由函数为 $f : q \rightarrow (D, \sigma, G)$ ，优化：

$$\max_{D, \sigma, G} \sum_{k=1}^K \text{rel}(t_k, \sigma(t_k)) + (1-\alpha) \sum_{(i,j) \in E} \text{compat}(\sigma_i, \sigma_j) \quad (1)$$

其中 $\text{rel}(\cdot)$ 度量子任务-技能相关性， $\text{compat}(\cdot)$ 度量技能间兼容性， $\alpha \in [0, 1]$ 控制相关性-兼容性权衡（在公式 6 中实例化）。联合优化公式 1 通常是不可行的，我们的级联流水线 (§4) 提供了一个可行的近似；SAD (§4.4) 通过将检索信号反馈到分解阶段进一步缩小近似差距。

4 方法：SKILLWEAVER

SKILLWEAVER 通过三个级联阶段实现组合式技能路由（图 1）。

4.1 阶段一：任务分解

给定复杂查询 q ，任务分解器使用指令微调的 LLM 生成有序原子子任务列表：

$$D(q) = \text{LLM}(p_{\text{sys}}, p_{\text{user}}(q)) = [t_1, \dots, t_K] \quad (2)$$

其中 p_{sys} 是指导模型将查询分解为子任务的系统提示，每个子任务恰好需要一项技能。输出约束为 JSON 字符串数组以确保可靠解析。

我们应用模型原生的对话模板（如 Qwen 模型的 ChatML）以确保正确的指令遵循。备用解析器处理 LLM 产生编号列表或其他非 JSON 格式的情况。

4.2 阶段二：技能检索

对于每个子任务 t_k ，使用双编码器架构从技能库中检索 top- m 个候选技能：

$$\text{cand}(t_k) = \text{top-}m_{s \in \mathcal{S}} \cos(E_q(t_k), E_s(s)) \quad (3)$$

其中 E_q 和 E_s 分别为查询编码器和技能编码器（共享参数），使用 BGE-large-en-v1.5 [Xiao et al., 2023] 实现。

技能表示。 我们研究两种编码策略：

- **仅元数据:** $\text{repr}(s) = n_s \oplus d_s$ （名称和描述）
- **包含正文:** $\text{repr}(s) = n_s \oplus d_s \oplus b_s[:L]$ （含截断正文）

其中 $\oplus$ 表示拼接， $L = 2000$ 字符。

嵌入向量经 L_2 归一化后使用 FAISS [Johnson et al., 2019] 建立索引以实现高效内积搜索。

4.3 阶段三：依赖感知的 DAG 规划

不同于假设固定顺序执行的先前工作，SKILLWEAVER 推断任务依赖并识别可并行化的子任务，生成有向无环图（DAG）作为执行计划。

依赖检测。 给定子任务 $[t_1, \dots, t_K]$ ，使用三种互补信号检测成对依赖：

1. **I/O 类型重叠:** 若 t_i 的预期输出类型与 t_j 的所需输入类型有交集，则推断 $t_i \rightarrow t_j$ 。
2. **生产者-消费者模式:** 精选关键词集识别输出产生动作（*generate, fetch, extract*）和输入消费动作（*parse, analyze, deploy*）。若 t_i 匹配生产者模式且 t_j 匹配消费者模式，则推断依赖。
3. **语言标记:** t_j 描述中出现“*then*”、“*after*”、“*based on*”等短语表示对先前步骤的显式依赖。若未检测到依赖，规划器保守地退回到顺序链式执行。

并行组分配。 给定依赖边 E ，通过拓扑层级分配（Kahn 算法）为每个子任务分配并行组 $g(t_k)$ ：

$$g(t_k) = \begin{cases} 0 & \text{if } \text{indeg}(t_k) = 0 \\ \max_{(t_i, t_k) \in E} g(t_i) + 1 & \text{otherwise} \end{cases} \quad (4)$$

具有相同组编号的子任务无互相依赖，可并行执行，将关键路径长度从 K （完全顺序）缩短至 DAG 深度 $d \leq K$ 。

Algorithm 1 技能感知分解 (SAD)**Require:** 查询 q , 技能库 $\mathcal{S}$, 检索器 R , 提示数量 H **Ensure:** 精化后的分解 $D^{(1)}(q)$

- 1: **第一轮 (原始):** $D^{(0)}(q) = [t_1, \dots, t_K] \leftarrow \text{LLM}(p_{\text{sys}}, q)$
- 2: **for** $k = 1$ to K **do**
- 3: $\text{cand}(t_k) \leftarrow R.\text{retrieve}(t_k, \text{top_k}=H)$
- 4: **end for**
- 5: $\mathcal{H} \leftarrow \text{top-}H \text{ skills from } \bigcup_k \text{cand}(t_k)$
- 6: **第二轮 (技能感知):** $D^{(1)}(q) \leftarrow \text{LLM}(p_{\text{sys}}, p_{\text{SAD}}(q, \mathcal{H}))$
- 7: **return** $D^{(1)}(q)$

多维兼容性。 我们定义上游技能 s_a 与下游候选 s_b 之间的四维兼容性函数:

$$\text{compat}(s_a, s_b) = \sum_{d \in \mathcal{D}} w_d \cdot \phi_d(s_a, s_b) \quad (5)$$

其中 $\mathcal{D} = \{\text{io}, \text{cat}, \text{tag}, \text{kw}\}$: ϕ_{io} 通过类型强制转换矩阵度量 I/O 类型兼容性; ϕ_{cat} 和 ϕ_{tag} 在类别和标签上计算 Jaccard 相似度; ϕ_{kw} 检测生产者-消费者关键词共现。权重设置为 (0.4, 0.2, 0.2, 0.2), 基于 30 个查询的验证集网格搜索; 结果对权重变化具有鲁棒性 (± 0.1 的变化使 CatR@1 变化 $< 0.5\%$)。

DAG 感知的技能选择。 最终每步技能结合检索相关性与 DAG 前驱兼容性:

$$\sigma(t_k) = \arg \max_{s \in \text{cand}(t_k)} \alpha \cdot \text{sim}(t_k, s) + (1 - \alpha) \cdot \bar{c}_k(s) \quad (6)$$

其中 $\bar{c}_k(s) = |\text{pred}(k)|^{-1} \sum_{i \in \text{pred}(k)} \text{compat}(\sigma_i, s)$ 对所有 DAG 前驱的兼容性取平均。

4.4 技能感知分解 (SAD)

我们实验的一个关键洞察是: LLM 分解器产生的通用子任务描述与技能元数据对齐不佳。我们提出技能感知分解 (SAD), 一种检索增强反馈环路, 使分解器获知可用技能。给定初始分解 $D^{(0)}(q)$ 和阶段二的检索结果 $\text{cand}(t_k)$, SAD 构造提示集 $\mathcal{H}$, 包含所有候选列表并集中的 top- H 个技能名称。分解器使用原始查询和 $\mathcal{H}$ 重新提示:

$$D^{(1)}(q) = \text{LLM}(p_{\text{sys}}, p_{\text{SAD}}(q, \mathcal{H})) \quad (7)$$

其中 p_{SAD} 将”可能相关的可用技能: $\mathcal{H}$ ”附加到分解提示中, 使分解器能产生与实际技能名称更好对齐的子任务描述。至关重要的是, 即使第一轮分解质量较差, SAD 仍然有效: 虽然不精确, 但各子任务描述仍包含与任务相关的关键词, 使检索器能浮现广泛相关的技能。由此产生的提示 (hint) 提供了用户查询语言与技能库命名约定之间的词汇桥梁, 使第二轮能生成与技能对齐的描述, 而不依赖于第一轮的质量。完整过程见算法 1。

类别	数量	示例
代码生成器	389	prd-generator, nextjs-dev
API 客户端	312	stripe-api, github-api
测试编写器	248	unit-test-gen, qa-suite
数据处理器	221	csv-parser, etl-pipeline
代码分析器	187	code-review, lint-checker
文档编写器	176	blog-writer, doc-gen
部署	142	k8s-deploy, ci-cd-setup
安全	118	vuln-scanner, audit-tool
可视化	97	chart-gen, dashboard-builder
搜索	96	web-search, code-search
+ 其余 7 个类别	共 609 个	

表 1: COMPSKILLBENCH 中前 10 个技能类别（共 17 个）。完整技能池包含来自 212 个仓库的 2,595 个技能。

5 基准测试：COMPSKILLBENCH

5.1 技能池构建

我们从 212 个公开 GitHub 仓库中收集采用 SKILL.md 规范格式的技能 [Anthropic, 2025]。原始收集得到 11,011 个技能。我们应用以下筛选流程：

- 去重**：合并同名技能，缩减至 7,681 个唯一技能。
- 质量过滤**：移除无描述或描述短于 10 字符的技能。
- 分类**：使用名称和描述字段上的关键词匹配，将技能分配到 17 个功能类别（表 1）。反关键词防止误分类（如”debug”技能从”deployment”中排除）。未匹配任何类别或匹配多个冲突类别的技能被排除。
- 最终技能池包含 **2,595** 个已分类技能。

5.2 查询生成

组合式查询通过将不同类别的技能组合为多步任务来生成：

难度级别。

- **简单**（117 个查询）：2 个技能，2 个类别
- **中等**（120 个查询）：3 个技能，3 个类别
- **困难**（63 个查询）：4-5 个技能，4-5 个类别

每个查询关联有真实子任务描述（源自实际技能描述）、真实技能 ID、所需类别和顺序执行序列。基准共包含 300 个查询，涉及 523 个唯一的真实技能。

文本重叠披露。 由于真实子任务描述源自技能元数据，子任务描述与技能库中的技能描述之间存在大量词汇重叠。自动分析显示真实子任务与对应技能描述的平均 ROUGE-L 为 0.845，平均 BLEU-1 为 0.980，其中 37.8% 为完全复制。这解释了近乎完美的 oracle 检索性能 ($R@1 = 99.5\%$)，意味着模板查询上的绝对检索分数应被视为上界估计。我们的人工撰写查询评估 (§7.4) 提供了互补评估，具有自然短语多样性，其中性能大幅下降（如 $CatR@1$: SAD 下 30.8% vs 54.2%），证实模板分数高估了真实性能。

5.3 评估指标

我们从三个粒度进行评估：

步骤级指标。

- **技能召回率 @ k ($R@k$)**：真实技能出现在 top- k 候选中的步骤比例。
- **类别召回率 @ k ($CatR@k$)**：正确类别中任何技能出现在 top- k 中的步骤比例。此宽松指标更具实用性，因为同一类别内的许多技能在功能上可互换。

链级指标。

- **链精确匹配 ($Chain_E$)**：所有步骤均选择了精确真实技能的查询比例。
- **链类别匹配 ($Chain_{cat}$)**：每个查询中选择了正确类别技能的步骤平均比例。

分解准确率 (DA)。 预测子任务数与真实值匹配的查询比例。DA 是严格的结构指标；一个有 3 个真实步骤的查询被分解为 4 个（包含一个额外有效中间步骤）将得到 $DA=0$ 。我们主要使用 DA 诊断分解粒度； $CatR@1$ 是主要质量指标。

6 实验设置

LLM 分解器。 我们比较五个开源指令微调模型，跨三个规模：Qwen2.5-7B-Instruct、Qwen2.5-14B-Instruct 和 Qwen2.5-72B-Instruct [Qwen Team, 2024]，Llama-3.1-8B-Instruct [Dubey et al., 2024]，以及 Mistral-7B-Instruct-v0.3 [Jiang et al., 2023]。生成使用温度 $\tau = 0.1$ ， $top_p = 0.9$ ，最大生成 256 个 token。我们通过三次不同随机种子运行验证低方差：vanilla 和 SAD 配置下 $CatR@1$ 标准差均 ≤ 0.003 。

检索器。 BGE-large-en-v1.5 [Xiao et al., 2023] (1024 维) 作为双编码器，使用 FAISS IndexFlatIP 进行精确内积搜索。除非另有说明，检索时设 $k = 10$ 。

基线方法。

- **单技能 (Single-Skill)**：不进行分解；使用完整查询检索一组技能并应用于所有步骤。代表当前的单技能路由方法。
- **LLM 直接选择 (LLM-Direct)**：不进行分解；向 LLM 提供查询和 top-50 检索技能的描述，直接选择使用哪些技能。测试上下文内选择能否替代显式分解。

方法	信号	DA	R@1	R@10	CatR@1	CatR@10	Chain _E	Chain _{cat}
基线方法								
单技能	Meta	–	0.000	0.029	0.211	0.698	0.000	0.216
单技能	Body	–	0.002	0.028	0.285	0.633	0.000	0.293
Oracle 分解	Meta	1.000	0.995	1.000	1.000	1.000	0.987	1.000
Oracle 分解	Body	1.000	0.990	1.000	0.998	1.000	0.970	0.998
完整流水线 (SKILLWEAVER)								
Qwen2.5-7B	Meta	0.360	0.005	0.034	0.339	0.649	0.000	0.316
Qwen2.5-7B	Body	0.370	0.008	0.029	<u>0.363</u>	<u>0.662</u>	0.000	<u>0.345</u>
Llama-3.1-8B	Meta	0.000	0.002	0.016	0.216	0.507	0.000	0.210
Mistral-7B	Meta	0.177	0.005	0.036	0.243	0.607	0.000	0.248
+ 技能感知分解 (§7.4)								
Qwen2.5-7B + SAD	Meta	<u>0.690</u>	<u>0.012</u>	<u>0.068</u>	<u>0.542</u>	<u>0.753</u>	<u>0.003</u>	<u>0.521</u>

表 2: 在 COMPSKILLBENCH 上的主要结果。**粗体**: 总体最优。下划线: 完整流水线中最优。R@ k : 技能召回率 @ k 。CatR@ k : 类别召回率 @ k 。Chain_E: 链精确匹配。Chain_{cat}: 链类别匹配。DA: 分解准确率。

- **Oracle 分解**: 使用真实子任务描述进行检索，建立分解质量的上界。

硬件。 7-14B 模型在单张 NVIDIA A100-80GB GPU 上运行。72B 模型使用四张 A100-80GB GPU，通过 device_map="auto" 自动张量并行。

7 实验结果

7.1 主要结果

表 2 展示了所有配置下的主要实验结果。

分解是瓶颈。 Oracle 分解达到 R@1 = 99.5% 和完美的类别准确率。最优 LLM 分解器 (Qwen2.5-7B) 仅达 CatR@1 = 36.3%，证实分解质量是主要瓶颈。

组合式路由优于单技能。 我们的完整流水线达到 CatR@1 = 33.9%，显著优于单技能基线 (21.1%)，验证了即使近似分解也有助于组合式路由。

元数据与正文表现相当。 仅元数据检索达到 oracle R@1 = 99.5% vs 正文感知的 99.0%。在完整流水线中，正文感知略有优势 (CatR@1: 36.3% vs 33.9%)，但编码成本更高，说明简洁的元数据已捕获关键辨别信号。

分解器	DA	CatR@1	Chain _{cat}
Qwen2.5-7B	0.360	0.339	0.316
Mistral-7B	0.177	0.243	0.248
Llama-3.1-8B	0.000	0.216	0.210
Oracle	1.000	1.000	1.000

表 3: 分解器比较（均使用仅元数据检索）。Qwen2.5-7B 达到最优的分解准确率和下游路由性能。

方法	难度	CatR@1	CatR@10
单技能	简单	0.201	0.774
	中等	0.267	0.667
	困难	0.146	0.675
Qwen + Meta	简单	0.214	0.637
	中等	0.367	0.658
	困难	0.410	0.646
Qwen + Body	简单	0.252	0.607
	中等	0.403	0.728
	困难	0.407	0.623

表 4: 按难度级别的性能分析。基于分解的路由在更难查询上展现更大优势。

7.2 分解器比较

表 3 比较了三个 LLM 分解器。Qwen2.5-7B 显著优于其他模型，分解准确率达 36.0%，而 Mistral 为 17.7%，Llama 为 0%。

过度分解是主要失败模式。 分解模式分析（图 2）揭示 Llama-3.1-8B 系统性地过度分解：中位输出为 9 个子任务，而真实分布在 2–3 处达峰。Llama 无单个分解匹配真实数量，解释了其 0% 的分解准确率。Mistral 呈现类似但程度较轻的趋势，Qwen 的分布与真实值最为接近。

7.3 难度分析

表 4 按难度级别分解结果。基于分解的流水线在较难查询上优势最大：对于困难查询（4–5 个技能），Qwen + Meta 达到 CatR@1 = 41.0%，而单技能基线为 14.6% (+181%)。这一模式表明，随着任务复杂度增加，分解变得越来越重要。

7.4 技能感知分解结果

SAD 消融实验。 表 5 展示了提示数量 H 的消融结果。DA 单调递增（36.0% 至 79.0%），但 CatR@1 在 $H=15$ 达峰（54.2%），随后在 $H=25$ 时下降（47.4%），揭示了噪声-对齐权衡。从 $H=0$

提示数 H	DA	CatR@1	Chain _{cat}
0 (原始)	0.360	0.339	0.316
5	0.457	0.468	0.445
10	0.537	0.519	0.498
15	0.690	0.542	0.521
25	0.790	0.474	0.451

表 5: SAD 消融实验 (Qwen2.5-7B)。DA 单调递增, 但 CatR@1 在 $H=15$ 处达峰, 揭示了噪声-对齐权衡。

模型	模式	DA	CatR@1	Chain _{cat}
Qwen2.5-7B	原始	0.360	0.339	0.316
	+SAD	0.690	0.542	0.521
Qwen2.5-14B	原始	0.280	0.297	0.280
	+SAD	0.443	0.385	0.377
Llama-3.1-8B	原始	0.000	0.216	0.210
	+SAD	0.217	0.339	0.323
Mistral-7B	原始	0.177	0.242	0.248
	+SAD	0.273	0.307	0.301
Qwen2.5-72B	原始	0.627	0.368	0.358
	+SAD	0.503	0.483	<u>0.518</u>

表 6: 跨模型 SAD 结果 ($H=15$)。SAD 在五个模型 (7–72B) 上均提升了路由性能。**粗体**: 最优。下划线: 次优。7B+SAD 超越了 10 倍规模的 72B 原始模型。

到 $H=5$ 的不成比例跳跃 (+38% CatR@1) 表明, 即使少量提示也能从根本上将分解器从自由形式转向技能感知生成, 额外提示的边际收益递减。

跨模型泛化。 表 6 显示 SAD 在不同模型家族中具有泛化性, CatR@1 提升从 +26.9% (Mistral) 到 +59.9% (Qwen-7B)。所有模型均使用 $H=15$ 以确保公平比较; 针对各模型单独调参可能取得更好结果。**Qwen2.5-7B+SAD (54.2%) 优于 72B 原始模型 (36.8%)**, 表明检索增强分解可超越 10 倍规模的模型。72B 模型的 SAD 将 DA 从 62.7% 降至 50.3%, 同时 CatR@1 提升至 48.3%。我们将此 DA 下降归因于提示诱导合并: 72B 模型本身具备细粒度分解能力 (原始平均 2.7 个子任务), 当技能提示暗示更粗粒度时会合并子任务, 产生更少但与技能更对齐的描述。分析证实 72B+SAD 平均每查询 2.3 个子任务 (vs 原始 2.7), 且保留的子任务与技能元数据的词汇重叠更高 (平均 ROUGE-L 0.72 vs 0.58)。我们推测 72B 的 CatR@1 低于 7B+SAD 反映了提示锚定效应: 较大模型倾向于逐字复制技能名称而非将提示用作词汇指引, 当复制的名称与真实技能不完全匹配时降低了检索对齐——这一模式在 14B 异常中也有观察 (附录 D)。14B 异常 (性能低于 7B) 源于过度分解和较低的 JSON 解析率; 详见附录 D。

策略	工具数	估计 Token	缩减比例
所有工具（朴素）	2,595	~1,038K	—
Top- k 平面检索	10	~4,000	99.6%
SKILLWEAVER（平均链）	2.9	~1,160	99.9%

表 7: 上下文窗口消耗 (~400 token/工具)。组合式路由将消耗减少两个数量级。

人工撰写查询泛化。 为补充模板评估，三名计算机科学研究研究生独立撰写了 55 个组合式查询（22 简单、23 中等、10 困难），遵循要求自然措辞且无法访问技能名称或描述的指引。SAD 将 CatR@1 从 18.8% 提升至 30.8% (+63.8%)。困难查询下降 (17.0% → 9.4%)：我们将此归因于提示稀释——困难查询需要 4–5 个跨多种类别的技能，因此 $H=15$ 的提示集不可避免地包含许多无关技能，误导了分解。此外，困难人工查询使用更间接的表述（如”搭建完整的 CI/CD 工作流”），加剧了难度。较低的绝对数值证实人工措辞多样性仍具挑战性。

7.5 消融实验：约束分解

为分离分解粒度（步骤数）与分解质量（子任务描述）的影响，我们进行消融实验：将 LLM 分解截断或填充以匹配真实步骤数。

约束分解达到 CatR@1 = 33.9%，与非约束流水线相同。这表明主要挑战不在于预测步骤数量，而在于生成与目标技能语义对齐的子任务描述。即使步骤数正确，LLM 生成的子任务描述可能过于通用或使用与技能元数据不同的术语，导致检索失配。

7.6 上下文窗口分析

表 7 量化了上下文节省：暴露所有 2,595 个技能消耗约 1.04M token，超过大多数 LLM 上下文限制；SKILLWEAVER 将每查询减少至 2–5 个技能 (>99% 缩减)。

8 分析

我们检查了 Qwen + Meta 流水线的 50 个失败案例：**过度分解** (36%)、**通用描述** (28%)、**词汇不匹配** (22%) 和 **不足分解** (14%)。Oracle R@1 (99.5%) 证实检索器质量；0.5% 的失败出现在技能具有近乎相同元数据的情况下。

9 讨论

分解差距。 Oracle 分解实现近乎完美的检索，但最优 LLM 分解器 (Qwen-7B) 仅达 CatR@1 = 33.9%；扩展至 72B 也仅达 36.8%。SAD 缩小了这一差距：**7B+SAD (54.2%) 超越 72B 原始模型 7.4 个百分点**，使用 $10\times$ 少的参数，跨模型结果确认 SAD 具有模型无关性。

模型	方法	CatR@1	Chain _{cat}
Qwen2.5-7B	原始流水线	0.339	0.316
	LLM 直接选择	0.396	—
	+SAD 流水线	0.542	0.521
Qwen2.5-72B	原始流水线	0.368	0.358
	LLM 直接选择	0.470	—
	+SAD 流水线	0.483	0.518

表 8: 分解 vs 直接选择。LLM 直接选择给模型提供 top-50 技能描述进行上下文内选择。**7B+SAD 优于 72B LLM 直接选择**，尽管参数少 10 $\times$ 。

分解 vs 直接选择。 一种替代方案是让 LLM 从 top-50 检索技能描述中直接选择。表 8 比较了此方法与我们的流水线。

72B LLM 直接选择达到 CatR@1 = 47.0%，但 **7B+SAD (54.2%) 仍超越之**，确认结构化分解配合检索反馈优于暴力上下文内选择。

元数据 vs 正文。 仅元数据检索与正文感知编码表现相当，说明组合式分解产生的目标查询自然地与简洁元数据对齐。

可扩展性。 FAISS 检索可扩展至数千技能（中位延迟 568ms, p95: 1.26s）。DAG 规划达到 80.0% 边检测率和 53.7% 功能一致性（附录 E）；组合式路由将上下文减少超过 99% (§7.6)。

10 结论

我们形式化了组合式技能路由并提出 SKILLWEAVER，一个分解-检索-组合框架，其中 SAD 在五个模型（7–72B）上将 CatR@1 提升 60%——配合 SAD 的 7B 模型优于不使用 SAD 的 72B 模型。在 COMPSKILLBENCH 上，oracle 检索近乎完美，确认分解是瓶颈。代码、数据和 COMPSKILLBENCH 将在论文接收后公开。

参考文献

- Anthropic. Agent skills specification. 2025. <https://docs.anthropic.com/en/docs/agents-and-tools/agent-skills>.
- Zehui Chen, Kuikun Du, Wenqian Chen, et al. Agentflan: Designing data and methods of effective agent tuning for large language models. *arXiv preprint arXiv:2403.12881*, 2024.
- Yu Du, Fangyun Fan, and Dingcheng Pi. Anytool: Self-reflective, hierarchical agents for large-scale api use. *arXiv preprint arXiv:2402.04253*, 2024.
- Abhimanyu Dubey et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

- Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings. *Advances in Neural Information Processing Systems*, 36, 2024.
- Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *Proceedings of the 39th International Conference on Machine Learning*, 2022.
- Yue Huang, Jiawen Shi, Yuan Li, Chenrui Fan, et al. Metatool benchmark for large language models: Deciding whether to use tools and which to use. *arXiv preprint arXiv:2310.03128*, 2024.
- Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020.
- Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed prompting: A modular approach for solving complex tasks. In *Proceedings of the International Conference on Learning Representations*, 2023.
- Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, et al. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- Jiarui Lu, Yinyao Yin, et al. Toolsandbox: A stateful, conversational, interactive evaluation framework for llm tool use capabilities. *arXiv preprint arXiv:2408.04682*, 2024.
- Jianmo Ni, Gustavo Hernandez Abrego, Noah Constant, Ji Ma, Keith B Hall, Daniel Cer, and Yinfei Yang. Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models. *Findings of the Association for Computational Linguistics: ACL 2022*, 2022.
- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.
- Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Jingqing Ruan, Yihong Chen, Bin Zhang, Zhiwei Xu, et al. Tptu: Task planning and tool usage of large language model-based ai agents. *arXiv preprint arXiv:2308.03427*, 2024.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2023.
- Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Taskbench: Benchmarking large language models for task automation. *arXiv preprint arXiv:2311.18760*, 2023.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.
- Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 2022.
- Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-pack: Packaged resources to advance general chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *Proceedings of the International Conference on Learning Representations*, 2023.
- Lifan Yuan, Yangyi Chen, Xingyao Wang, et al. Craft: Customizing llms by creating and retrieving from specialized toolsets. In *Proceedings of the International Conference on Learning Representations*, 2025.
- YanZhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang Yu, JiHuai Zhu, Yong Wu, Tianze Xu, Baohua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. Skillrouter: Retrieve-and-rerank skill selection for llm agents at scale. *arXiv preprint arXiv:2603.22455*, 2025.
- Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*, 2022.

Yuchen Zhuang, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. Toolqa: A dataset for llm question answering with external tools. *Advances in Neural Information Processing Systems*, 36, 2024.

局限性

我们的基准使用模板生成查询，文本重叠较高（ROUGE-L=0.845）；虽然 55 个人工撰写查询在简单/中等难度上显示一致的 SAD 增益，但困难查询（4–5 个技能）性能下降。DAG 规划器的并行化能力缺乏偏序真实值评估。我们的评估聚焦检索；完整端到端评估（实际技能执行）和基于学习的依赖预测仍为未来工作。SAD 需要两轮 LLM 推理，分解延迟约增加一倍（7B 模型在 A100 上共约 1.6 秒）。我们仅评估开源模型；闭源模型可能实现更高的原始分解质量，但 SAD 是正交方法，可进一步提升其性能。我们假设子任务与技能之间为一对一映射；放宽为多对多映射是未来工作。困难子集（63 个模板查询、10 个人工查询）统计效力有限；在困难人工查询上观察到的 SAD 下降可能部分反映采样噪声。

伦理声明

本工作仅使用公开可用的开源技能仓库，不涉及人类受试者或个人数据。我们鼓励在人工监督下负责任地部署技能路由系统。

A 类别分类体系

COMPSKILLBENCH 中的 17 个功能类别为：code_generator、api_client、test_writer、data_processor、code_analyzer、writer、deployment、security、visualizer、search、refactorer、file_converter、translator、database、monitoring、designer、git_workflow。

B 分解分布

图 2 展示了各 LLM 分解器预测的子任务数量分布与真实值的比较。

C Top- k 敏感性

使用 oracle 分解， $R@1 = 99.5\%$ 在 $k = 1$ 时即可达到， $R@k$ 在 $k = 3$ 时达到 100%，表明检索器具有很高的精确度。

D 14B 异常分析

Qwen2.5-14B 表现低于 7B 变体的反直觉结果值得进一步检验。分解模式分析揭示两个成因。首先，14B 原始模型平均每查询产生 5.3 个子任务（vs 7B 的 2.9），42.7% 的查询生成超过 5 个子任务——严重的过度分解碎片化了检索信号。其次，14B 原始输出仅 71.3% 可解析为有效 JSON

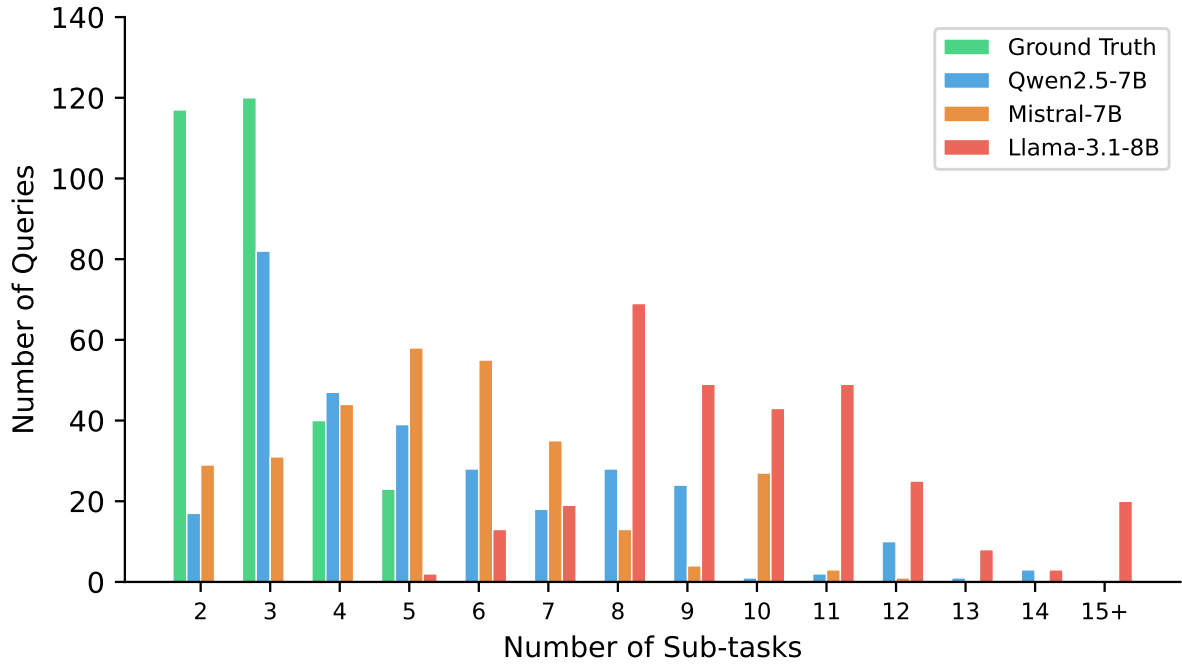


图 2: 各分解器预测的子任务数量分布。Llama-3.1-8B 系统性地过度分解（中位 9 个子任务），而 Qwen2.5-7B 与真实分布最为接近（峰值在 2–3）。

(vs 7B 的 89.3%)，表明该规模下对结构化输出格式的遵循较弱。使用 SAD 后，14B 的 JSON 解析率提升至 85.0%，但 44.7% 的查询退化为单子任务回退，表明模型逐字复制技能名称而非产生真正的分解。这些发现表明更大的模型并不一定在结构化分解方面表现更好。

E DAG 规划器试验评估

表 9 报告了在 20 个查询的分层试验集（8 简单、8 中等、4 困难）上使用 Qwen2.5-7B + SAD ($H=15$) 的 DAG 规划器评估结果。

难度	n	步骤准确率	边/查询	一致性	延迟 (ms)
简单	8	0.75	0.8	0.56	849
中等	8	0.75	1.6	0.56	564
困难	4	0.50	2.5	0.44	889
总计	20	0.70	1.4	0.54	743

表 9: DAG 规划器试验评估。步骤准确率：预测步骤数与真实值匹配的查询比例。边/查询：每查询检测到的平均依赖边数。一致性：功能一致性（相邻技能对具有兼容 I/O 类别的比例）。80% 的查询至少检测到一条依赖边。中位延迟 568ms (p95: 1,261ms)。不同难度间的延迟变化反映小样本量 ($n=4-8$) 而非系统性趋势。

F SAD 提示模板

系统提示（原始和 SAD 共用）。

```
You are a task decomposition assistant. Given a complex user query, break it
down into atomic sub-tasks, each requiring exactly one tool or skill. Output
a JSON array of strings. Each string should be a concise, actionable sub-task
description.
```

原始用户提示。

```
Decompose the following query into atomic sub-tasks:
{query}
```

SAD 用户提示（第二轮）。

```
Decompose the following query into atomic sub-tasks.
Available skills that may be relevant: {hint_list}
Query: {query}
```

其中 {hint_list} 是第一轮检索到的 top- H 个技能名称的逗号分隔列表（见算法 1）。